Online Restruant Ordering System

Techinal Manual

Devloped by:

Cobra.py

ITSC - 3155

Overview

We built the Restaurant Management API built using FastAPI and uses a RESTful architecture. Implementation does not define all endpoint in one file but instead the application separates functionality into separate controller modules, each being responsible for a specific feature

Architecture Flow

Client → FastAPI Routes → Database (SQLAlchemy) → MySQL

^ main.py (router registration)

Design Approach

- main.py acts as a central entry poiny
- All endpoints are defined inside controller or router files
- Each module contains its own routes
- Routers are registered in main,pu using include_router()

Key Components

main.py

- Initializes FastAPI app
- Configures middleware (CORS)
- Loads database models
- Registers all routers using:

app.include_router(products.router)

Models (models.py)

- Define database tables using SQLAlchemy
- Represent entities such as Orders, Products, Payments, etc.

Schemas (schemas.py)

- Define request and response data formats
- Ensure validation using Pydantic

Controllers (routers)

- products.py -> Menu management
- orders.py -> Order management
- Order_details.py -> Order item handliong
- payments.py -> Payment processing
- reviews.py -> Customer feedback
- Order_tracking.py -> Tracking System

- Customer_orders.py -> Order Management
- analytings.py -> Reporting
- promotions.py -> Discounts

Database (database.py)

- Manages database connection
- Provides session dependency (get_db)

CRUD Operations Implementation

CRUD operations are implemented inside each Schema which is imported into controller files

Example: Products.py

```python
@router.post( path: "/", response_model=ProductRead, status_code=status.HTTP_201_CREATED)  ndmerritt +1
def create_menu_item(product_in: ProductCreate, db: Session = Depends(get_db)):
    new_product = Product(**product_in.model_dump())
    db.add(new_product)
    db.commit()
    db.refresh(new_product)
    return new_product


@router.put("/{product_id}")  ndmerritt +1
def update_menu_item(product_id: int, updates: ProductUpdate, db: Session = Depends(get_db)):
    product = db.query(Product).filter(Product.id == product_id).first()
    if not product:
        raise HTTPException(status_code=404, detail="Product not found")

    update_data = updates.model_dump(exlude_unset=True)

    for key, value in updates.items():
        setattr(product, key, value)

    db.commit()
    db.refresh(product)
    return product
```

```python
@router.delete("/{product_id}")  ndmerritt
def delete_menu_item(product_id: int, db: Session = Depends(get_db)):
    product = db.query(Product).filter(Product.id == product_id).first()
    if not product:
        raise HTTPException(status_code=404, detail="Product not found")

    product.is_active = False
    db.commit()
    return {"message": f"Product {product_id} has been deactivated from the menu"}
```

```python
@router.get("/{product_id}")  ndmerritt
def get_product_details(product_id: int, db: Session = Depends(get_db)):
    product = db.query(Product).filter(Product.id == product_id).first()
    if not product:
        raise HTTPException(status_code=404, detail="Product not found")
    return product
```

```python
# 🔹 Shared fields
class ProductBase(BaseModel):  2 usages  Cooper Adams
    name: str
    price: float
    description: Optional[str] = None
    category: Optional[str] = None
    stock: Optional[int] = None


# 🔹 Create
class ProductCreate(ProductBase):  2 usages  Cooper Adams
    pass


# 🔹 Update (all optional)
class ProductUpdate(BaseModel):  2 usages  Cooper Adams
    name: Optional[str] = None
    price: Optional[float] = None
    description: Optional[str] = None
    category: Optional[str] = None
    stock: Optional[int] = None


# 🔹 Read (response)
class ProductRead(ProductBase):  2 usages  Cooper Adams
    id: int

    class Config:  Cooper Adams
        from_attributes = True
```

Key Concepts

APIRouter

- Used to separate endpoints into modules
- Improves readability and scalability

Dependency Injection

db : Session = Depends(get_db)

- Automatically provides database connection

SQLAlchemy ORM

- Handles all database operations

Pydantic Schemas

- Validate incoming request data
- Define response structure

Advantages

- Modular and organized
- Easy to expand
- Separates functionality by domain
- Matches FastAPI practices in the industry

Summary

API implements CRUD operations through a controller system where each module manages its own endpoints and logic, main.py regulates all modules. And the system remains scalable and maintainable